Generative Adversarial Networks (GANs) – Section 4: Training Process, Challenges & Solutions

Introduction

Understanding GAN mathematics and architecture is necessary but not sufficient. In practice, GAN training is notoriously difficult – it is brittle, sensitive to hyperparameters, and prone to catastrophic failure modes that can appear without warning. Researchers spent years discovering why these failures happen and developing principled solutions.

This section is the practitioner's survival guide. We go deep into the training loop mechanics, diagnose every major failure mode with mathematical precision, and equip you with proven solutions that work in production.

In this section, we will cover:

- **The training loop in depth** – the exact sequence of operations and why order matters
- **Mode collapse** – the most common GAN failure and its mathematical roots
- **Training instability** – oscillation, divergence, and the Nash equilibrium problem
- **The discriminator dominance problem** – what happens when D becomes too powerful
- **Practical stabilization techniques** – spectral normalization, minibatch discrimination, experience replay
- **Progressive growing** – the architectural training strategy behind state-of-the-art image GANs
- **Monitoring and diagnostics** – how to tell if your GAN is training well or collapsing
- **Code** – a complete, production-grade training loop with all stabilization techniques

---

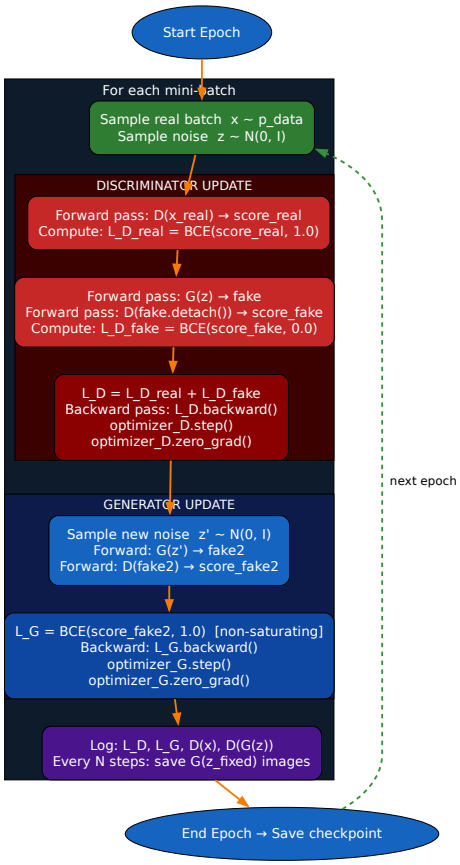
1. The Training Loop in Depth

What It Is

The GAN training loop is a carefully orchestrated sequence of alternating optimization steps. Unlike standard supervised learning where a single model minimizes a single loss, GANs require **simultaneous optimization of two networks with opposing objectives**. The order of operations, the number of steps per network, and the handling of gradient flow between the two networks all matter.

> **Real-World Analogy – Two Athletes Training Together:**
>
> Imagine two martial artists training against each other. They alternate: one attacks while the other defends, then they switch. If one trains 100 rounds while the other rests, the rested one becomes overwhelmed – they can no longer give useful feedback. If they both train together in short balanced bursts, both improve steadily. The GAN training loop works the same way: balanced alternating updates keep both networks in the productive zone where each can challenge the other.

The Exact Step Sequence



Why Each Step Is Ordered This Way

Why update D before G? The Discriminator's job is to provide a gradient signal to the Generator. If the Generator is updated first, it receives a signal from an outdated Discriminator – one that hasn't yet seen the Generator's latest outputs. Updating D first ensures G receives the most current, most informative gradient.

Why use `.detach()` when computing $D(\text{fake})$ for the D update? When training the Discriminator, we want to update D's weights only – not G's. Without `.detach()`, PyTorch tracks gradients all the way back through the Generator, and the D backward pass would incorrectly modify G's weights. `.detach()` cuts this connection.

Why sample fresh noise for the G update? Using the same fake images computed during the D update would mean the Generator updates based on stale Discriminator scores (computed before the D update step). Fresh noise ensures G is evaluated by the newly-updated Discriminator.

Why `zero_grad()` after each optimizer step? PyTorch accumulates gradients by default. Without zeroing, gradients from the current step add to gradients from all previous steps, causing wildly incorrect updates.

Key Metrics to Track Each Step

Metric	Symbol	Healthy Range	Warning Sign
Discriminator loss	$\mathcal{L}_D$	Near $\ln 2 \approx 0.693$	Near 0: D too powerful

Metric	Symbol	Healthy Range	Warning Sign
Generator loss	$\mathcal{L}_G$	Near $\ln 2 \approx 0.693$	Rising continuously: mode collapse
D score on real	$D(x)$	Near 0.5 at equilibrium	Stuck near 1.0: D overconfident
D score on fake	$D(G(z))$	Near 0.5 at equilibrium	Stuck near 0.0: G not fooling D

---

2. Mode Collapse – The Most Dangerous GAN Failure

What It Is

Mode collapse occurs when the Generator learns to produce only a small subset of the possible outputs – often just one or a few "modes" of the data distribution – even though the real data has many diverse modes. The Generator finds a small set of outputs that reliably fool the Discriminator and exploits them, ignoring the rest of the data distribution.

> **Real-World Analogy – A Student Who Only Studies One Topic:**
>
> Imagine an exam covers 10 topics. A student discovers that question 3 always appears and memorizes only that answer. They pass every practice exam (the discriminator is fooled) but know almost nothing about the subject (the Generator only covers one mode). A good teacher would notice the pattern and change the questions – but if the teacher-student dynamic locks into a cycle, the student never has to learn the full curriculum.
>
> In GAN terms: the Generator "memorizes" a few safe outputs. The Discriminator learns to detect this pattern and forces the Generator to shift to different outputs. But instead of covering all modes, the Generator shifts to a new single mode – and the cycle continues. This cycling between single modes is called **mode hopping**.

The Mathematical Cause of Mode Collapse

Mode collapse happens because the Generator's optimization landscape contains **sharp local minima** associated with individual modes. Consider a real data distribution with K modes (clusters):

$$p_{data}(x) = \sum_{k=1}^{K} \pi_k \cdot \mathcal{N}(x; \mu_k, \Sigma_k)$$

The Generator is a function $G : \mathcal{Z} \to \mathcal{X}$ mapping the noise space to the data space. For the Generator to cover all K modes, the noise space must be divided such that different regions of $\mathcal{Z}$ map to different modes.

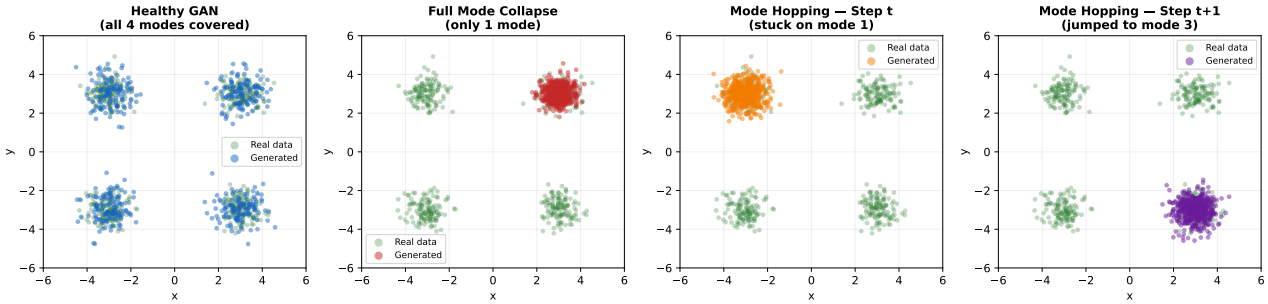
The problem: During training, the Generator may find that mapping **all** of $\mathcal{Z}$ to a single mode μ_1 produces a loss that is locally better than spreading across all modes. The Discriminator then learns mode μ_1 is fake. The Generator then collapses to μ_2. This **mode hopping** cycle never allows the Generator to cover all modes simultaneously.

Formally, the Generator is optimizing:

$$\min_G \mathbb{E}_{z \sim p_z}[-\log D(G(z))]$$

This objective contains no term that **penalizes low diversity** in $G(z)$. The Generator is never directly told "generate varied outputs." It only has to fool D – and fooling D with a single convincing output satisfies the objective just as well as covering all modes.

Visualizing Mode Collapse



Reading the plots:

- **Healthy GAN (left):** Generated samples (blue) cover all 4 green clusters – full diversity
- **Full collapse (second):** All generated samples pile into one mode – 75% of the real distribution is completely ignored
- **Mode hopping (right two):** The Generator jumps between single modes across training steps – never covering all modes simultaneously. This cycling behavior is often visible as periodic spikes in the Generator loss curve

Solutions to Mode Collapse

Solution 1 – Minibatch Discrimination **Idea:** Give the Discriminator the ability to compare samples within a mini-batch, not just evaluate them independently. If all samples in a batch look nearly identical (collapse), the Discriminator can detect this and penalize it.

How it works: For each sample in the batch, compute pairwise distances to all other samples. Append these distance statistics as additional features to the Discriminator's input. If distances are all near zero (collapse), the Discriminator receives a signal that something is wrong.

$$o(x_i) = \sum_{j=1}^{B} \exp(-\|f(x_i) - f(x_j)\|_1)$$

where $f(x)$ is an intermediate feature vector and B is the batch size. Small $o(x_i)$ means all samples are similar – a collapse signal.

Solution 2 – Unrolled GANs **Idea:** Instead of updating the Generator against the current Discriminator $D^{(t)}$, update it against $D^{(t+k)}$ – the Discriminator after k steps of update. This makes the Generator "think ahead" and not exploit short-term weaknesses.

Solution 3 – WGAN / WGAN-GP As we saw in Section 3, the Wasserstein loss penalizes low coverage of the real distribution more evenly. The Critic must score samples globally, not just locally – making it harder for the Generator to exploit a single mode.

---

3. Training Instability – Oscillation and Divergence

What It Is

Even without mode collapse, GAN training can become **unstable** – losses oscillate wildly, images flicker between good and bad quality, or training diverges entirely (losses go to infinity or NaN).

> **Real-World Analogy – Two Arms Wrestling Through a Table:**
> Imagine two arm wrestlers locked in a perfect stalemate – neither can win, but both keep pushing. If one gets slightly stronger and lunges forward, the other reactively pulls back harder, overshooting the equilibrium. They oscillate around the stalemate point, never settling. GANs face the exact same problem: finding the Nash equilibrium in a continuous, high-dimensional space is difficult because gradient descent doesn't guarantee convergence to equilibrium in multi-player games.

The Nash Equilibrium Problem

Standard gradient descent is guaranteed to converge to a local minimum for a single objective. For two objectives, there is **no such guarantee**. Each player follows the gradient of their own loss, but this can create **rotational dynamics** that orbit the equilibrium rather than converging to it.

A Simple Example: Consider two scalar variables θ_G and θ_D with the loss:

$$V(\theta_G, \theta_D) = \theta_G \cdot \theta_D$$

Gradient updates:

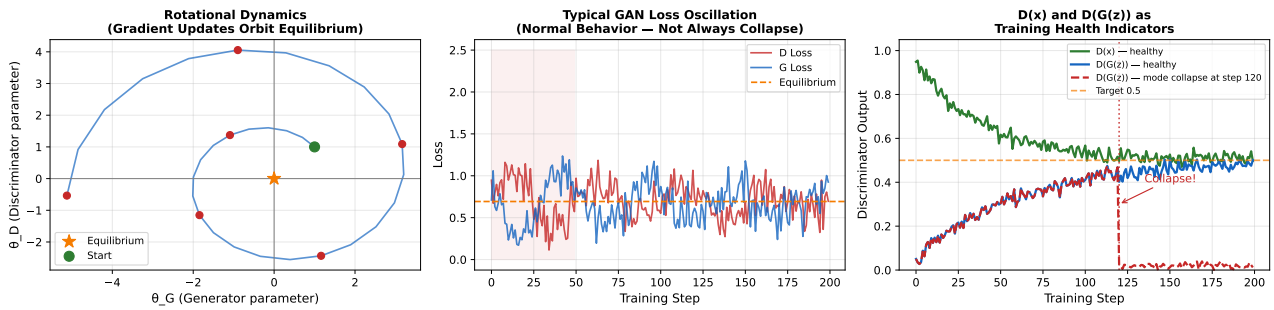
$$\theta_D \leftarrow \theta_D + \eta \cdot \frac{\partial V}{\partial \theta_D} = \theta_D + \eta \cdot \theta_G$$

$$\theta_G \leftarrow \theta_G - \eta \cdot \frac{\partial V}{\partial \theta_G} = \theta_G - \eta \cdot \theta_D$$

Numerical trace (starting at $\theta_G = 1, \theta_D = 1$, $\eta = 0.5$):

Step	θ_G	θ_D	$V = \theta_G \cdot \theta_D$
0	1.0	1.0	1.0
1	$1.0 - 0.5(1.0) = 0.5$	$1.0 + 0.5(1.0) = 1.5$	0.75
2	$0.5 - 0.5(1.5) = -0.25$	$1.5 + 0.5(0.5) = 1.75$	-0.44
3	$-0.25 - 0.5(1.75) = -1.125$	$1.75 + 0.5(-0.25) = 1.625$	-1.828
4	$-1.125 - 0.5(1.625) = -1.9375$	$1.625 + 0.5(-1.125) = 1.0625$	-2.059

The equilibrium is $(0, 0)$ but the updates spiral outward – this is **rotational divergence**, a failure mode unique to two-player games.



Reading the plots:

- **Plot 1 (Rotational dynamics):** The gradient update trajectory spirals outward from the equilibrium (orange star) rather than converging to it – this is the fundamental instability of two-player gradient descent
- **Plot 2 (Loss oscillation):** Some oscillation around equilibrium is normal and expected. The key sign of health is that oscillations **decay over time** and hover near $\ln 2 \approx 0.693$
- **Plot 3 (D scores as diagnostic):** In a healthy run, $D(x)$ and $D(G(z))$ both converge to 0.5. A sudden drop of $D(G(z))$ to near 0 (red dashed line, step 120) is the clearest signal of mode collapse or Generator failure

---

4. The Discriminator Dominance Problem

What It Is

If the Discriminator becomes too powerful too quickly – perfectly separating real from fake with 100% accuracy – the Generator receives **gradients that are essentially zero**, and it cannot learn.

This is related to, but distinct from, the vanishing gradient problem in Section 3. Here the cause is not the loss function formula – it is the **training dynamics** where D simply outpaces G.

Why It Happens Mathematically

When the Discriminator is perfect ($D(x_{real}) \approx 1$, $D(G(z)) \approx 0$):

$$\mathcal{L}_G = -\log D(G(z)) \approx -\log(0.001) \approx 6.9$$

The Generator loss is very high – which sounds like it should drive learning. But look at the gradient:

$$\frac{\partial \mathcal{L}_G}{\partial D(G(z))} = \frac{-1}{D(G(z))} \approx \frac{-1}{0.001} = -1000$$

This is a massive gradient – but it flows backward through the Discriminator's weights, not through a useful path for the Generator. By the time it reaches the Generator's early layers (through many layers of backpropagation), the **chain rule product** of many small values reduces it to near zero.

More precisely: a perfect Discriminator has learned to classify based on features that the Generator has not yet learned to produce. The gradient it sends to the Generator points in a useless direction – "don't produce these specific bad pixels" – rather than "here's what real features look like."

Solutions to Discriminator Dominance

Solution 1 – Spectral Normalization **Spectral normalization** constrains each weight matrix W in the Discriminator by dividing it by its **spectral norm** (the largest singular value):

$$W_{SN} = \frac{W}{\sigma(W)}$$

Symbol definitions:

Symbol	Meaning
W	Original weight matrix of a layer
$\sigma(W)$	Spectral norm of W – the largest singular value of W
W_{SN}	Spectrally normalized weight – guaranteed to have spectral norm ≤ 1

Why this works: Constraining the spectral norm of each layer bounds the Lipschitz constant of the entire network, preventing the Discriminator from becoming too sharp. This is a **soft constraint** – unlike weight clipping, it doesn't limit what functions the Discriminator can represent, only how sharply it can change.

Intuition: The spectral norm measures the maximum "stretch factor" a weight matrix can apply to any input vector. If $\sigma(W) = 1$, the matrix cannot amplify any input direction more than $1\times$ – it's a balanced transformation. Spectral normalization ensures the Discriminator remains a well-scaled function rather than a sharp decision boundary.

Numerical Example – Spectral Normalization:

Consider a 2×2 weight matrix:

$$W = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}$$

Step 1 – Compute the singular values via SVD (or power iteration):

The singular values of W are the square roots of the eigenvalues of $W^T W$:

$$W^T W = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}^T \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 10 & 5 \\ 5 & 5 \end{bmatrix}$$

Eigenvalues of $W^T W$: solving $\det(W^T W - \lambda I) = 0$:

$$(10 - \lambda)(5 - \lambda) - 25 = 0$$

$$\lambda^2 - 15\lambda + 25 = 0 \implies \lambda = \frac{15 \pm \sqrt{225 - 100}}{2} = \frac{15 \pm \sqrt{125}}{2}$$

$$\lambda_1 = \frac{15 + 11.18}{2} \approx 13.09, \quad \lambda_2 = \frac{15 - 11.18}{2} \approx 1.91$$

Step 2 – Spectral norm $= \sigma(W) = \sqrt{\lambda_1} \approx \sqrt{13.09} \approx 3.62$

Step 3 – Normalized weight:

$$W_{SN} = \frac{W}{\sigma(W)} = \frac{1}{3.62}\begin{bmatrix}3 & 1\\ 1 & 2\end{bmatrix} = \begin{bmatrix}0.829 & 0.276\\ 0.276 & 0.552\end{bmatrix}$$

The normalized matrix has spectral norm exactly 1 – the Discriminator layer is bounded.

Solution 2 – Experience Replay **Idea:** Maintain a buffer of previously generated fake images and occasionally train the Discriminator on these historical fakes instead of only current fakes. This prevents the Discriminator from overfitting to the Generator's current output distribution.

Why it works: If the Generator has previously produced mode A and now produces mode B, a Discriminator trained only on current fakes forgets mode A. When the Generator revisits mode A later, the Discriminator is fooled. With experience replay, the Discriminator remembers all past Generator states and cannot be exploited by revisiting old patterns.

Solution 3 – Update Ratio Tuning Instead of 1 Discriminator step per Generator step, use $n_D : 1$ ratio:

- **More D steps ($n_D > 1$):** Gives the Generator a stronger, more refined gradient signal. Required in WGAN (typically $n_D = 5$) but can cause D dominance in standard GANs
- **Fewer D steps ($n_D < 1$, i.e., skip D sometimes):** Prevents D from getting too far ahead of G. Useful when D loss drops near 0 while G loss keeps rising

---

5. Spectral Normalization – Full Implementation

Reading the diagram: Spectral normalization is computed **on-the-fly during each forward pass**. The raw weight W is maintained as a learnable parameter. Each forward pass estimates $\sigma(W)$ via one power iteration step and divides W by this estimate. Gradients flow back to the raw (unnormalized) W, which the optimizer updates. On the next forward pass, $\sigma(W)$ is re-estimated and re-applied. This is computationally inexpensive – one power iteration step per layer per forward pass.

---

6. Progressive Growing – Training GANs at High Resolution

What It Is

Progressive Growing of GANs (ProGAN), introduced by Karras et al. (2018) from NVIDIA, is the architectural training strategy that enabled photorealistic 1024×1024 face generation. The key insight is: **instead of training a full-resolution GAN from scratch, start with a tiny 4×4 GAN and progressively add new layers to both G and D as training stabilizes**.

> **Real-World Analogy – Learning to Draw by Zooming In:**
> A professional artist doesn't start a portrait by immediately trying to paint every pore and hair. They start with a rough sketch – just the major shapes and proportions. Once those are right, they add details layer by layer: rough shading, then

face structure, then features, then fine details. Each layer builds on a stable foundation. Progressive GAN training works identically: master the big picture (4×4) before worrying about fine details (1024×1024).

Why Progressive Growing Works

Training a high-resolution GAN from scratch faces two problems:

1. **Vanishing gradients across many layers:** A deep Generator has many layers for gradients to propagate through. Early in training, these gradients can vanish before reaching the early layers
2. **Unstable early training:** At 1024×1024, the Generator must simultaneously learn global structure (faces have two eyes) and local texture (individual hair strands). This is an extremely hard joint optimization

Progressive growing solves both by:

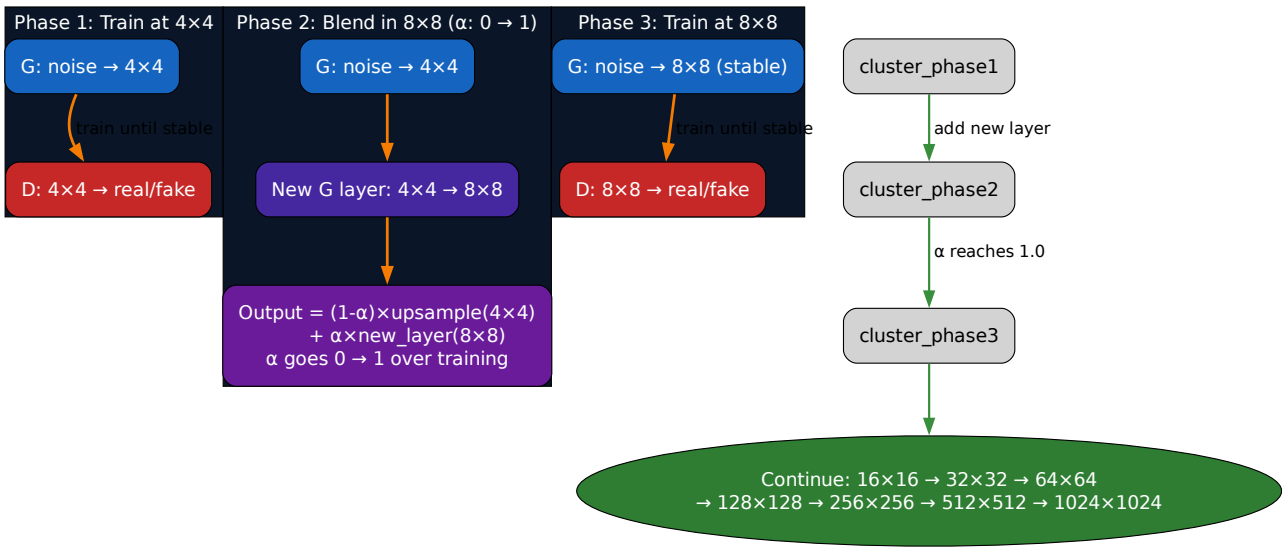
- Starting simple: a 4×4 image has only 48 pixels – the Generator must learn only the most basic face structure (roughly round, skin-colored blob)
- Adding complexity gradually: each new layer added to G and D is **smoothly blended in** using a mixing weight α that transitions from 0 to 1
- Building stable foundations: by the time high-resolution layers are added, the lower layers already produce correct low-frequency content

The Alpha Blending Transition

When adding a new layer, the output is blended:

$$\text{output} = (1 - \alpha) \cdot \text{upsampled_previous} + \alpha \cdot \text{new_layer_output}$$

where α transitions from 0 (purely previous resolution) to 1 (purely new layer) over thousands of training steps.



---

7. Code – Complete Production Training Loop

Full Training Loop with All Stabilization Techniques

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import torchvision.utils as vutils
import random
from collections import deque

# -- Spectral Normalization Discriminator --------------------------
# PyTorch provides spectral normalization as a weight hook
from torch.nn.utils import spectral_norm

class StableDiscriminator(nn.Module):
    """
    Discriminator with spectral normalization on every Conv layer.
    Prevents D from becoming too powerful by constraining Lipschitz
 ↪  constant.
    No BatchNorm - replaced with spectral norm for Lipschitz control.
    """
    def __init__(self, n_features=64):
        super().__init__()
        nf = n_features

        self.main = nn.Sequential(
            # spectral_norm() wraps the layer: during forward, W is
             ↪  divided by σ(W)
            spectral_norm(nn.Conv2d(3, nf, 4, 2, 1, bias=False)),
            nn.LeakyReLU(0.2, inplace=True),

            spectral_norm(nn.Conv2d(nf, nf*2, 4, 2, 1, bias=False)),
            # No BatchNorm with spectral norm - they interfere
            nn.LeakyReLU(0.2, inplace=True),

            spectral_norm(nn.Conv2d(nf*2, nf*4, 4, 2, 1, bias=False)),
            nn.LeakyReLU(0.2, inplace=True),

            spectral_norm(nn.Conv2d(nf*4, nf*8, 4, 2, 1, bias=False)),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Flatten(),
            spectral_norm(nn.Linear(nf*8*4*4, 1)),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.main(x)
```

```python
# -- Experience Replay Buffer ---------------------------------------
class ReplayBuffer:
    """
    Stores previously generated fake images.
    Returns mix of current and historical fakes to train Discriminator.
    Prevents D from forgetting previous G outputs → reduces mode cycling.
    """
    def __init__(self, max_size=500):
        self.buffer = deque(maxlen=max_size)  # Auto-evicts oldest when
        ↪  full

    def push(self, imgs):
        # Store each image individually
        for img in imgs.detach().cpu():
            self.buffer.append(img)

    def sample(self, batch_size, current_fake, use_history_prob=0.5):
        """
        With probability use_history_prob, replace some current fakes
        with historical ones. Otherwise return current fakes unchanged.
        """
        if len(self.buffer) < batch_size or random.random() >
        ↪  use_history_prob:
            return current_fake

        # Sample from buffer to fill half the batch
        n_historical = batch_size // 2
        historical = random.sample(self.buffer, n_historical)
        historical = torch.stack(historical).to(current_fake.device)

        # Mix historical and current fakes
        mixed = torch.cat([historical, current_fake[n_historical:]],
↪   dim=0)
        return mixed


# -- Training Loop with All Stabilizations -------------------------
def train_stable_gan(G, D, dataloader, n_epochs, device,
                     use_label_smooth=True,
                     use_replay=True,
                     n_critic_steps=1):
    """
    Production-grade GAN training loop.
    Includes: label smoothing, experience replay, loss monitoring,
              gradient norm tracking, periodic sample saving.
    """
    criterion  = nn.BCELoss()
    opt_G = optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
    opt_D = optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))

    replay_buffer = ReplayBuffer(max_size=500) if use_replay else None
```

```python
    # Fixed noise for consistent progress visualization across epochs
    fixed_noise = torch.randn(64, 100, device=device)

    history = {'d_loss': [], 'g_loss': [], 'd_real': [], 'd_fake': []}

    for epoch in range(n_epochs):
        epoch_d_loss, epoch_g_loss = [], []
        epoch_d_real, epoch_d_fake = [], []

        for i, (real_imgs, _) in enumerate(dataloader):
            real_imgs = real_imgs.to(device)
            B = real_imgs.size(0)

            # -- Labels with optional one-sided smoothing ----------
            real_label = 0.9 if use_label_smooth else 1.0  # Soft real
↪   label
            fake_label = 0.0                                # Hard fake
↪   label

            # -- n_critic_steps D updates per G update -------------
            for _ in range(n_critic_steps):
                D.zero_grad()

                # Real data loss
                real_out = D(real_imgs)
                d_real_loss = criterion(real_out,
                    torch.full_like(real_out, real_label))

                # Fake data - with optional experience replay
                noise = torch.randn(B, 100, device=device)
                with torch.no_grad():           # Don't track G grads here
                    fake_imgs = G(noise)

                if replay_buffer is not None:
                    replay_buffer.push(fake_imgs)
                    # Mix: 50% chance to use historical fakes
                    fake_for_D = replay_buffer.sample(B, fake_imgs,
                                                      use_history_prob=0.5)
                else:
                    fake_for_D = fake_imgs

                fake_out = D(fake_for_D.detach())
                d_fake_loss = criterion(fake_out,
↪   torch.zeros_like(fake_out))

                d_loss = d_real_loss + d_fake_loss
                d_loss.backward()
                opt_D.step()

            # -- Generator update ----------------------------------
            G.zero_grad()
```

```
            noise2    = torch.randn(B, 100, device=device)
            fake_imgs2 = G(noise2)
            gen_out    = D(fake_imgs2)

            # Non-saturating loss: maximize log D(G(z)) via real labels
             ↪  target
            g_loss = criterion(gen_out, torch.ones_like(gen_out))
            g_loss.backward()
            opt_G.step()

            # -- Track metrics ------------------------------------
            with torch.no_grad():
                epoch_d_loss.append(d_loss.item())
                epoch_g_loss.append(g_loss.item())
                epoch_d_real.append(real_out.mean().item())
                epoch_d_fake.append(gen_out.mean().item())

        # -- Epoch summary ----------------------------------------
        avg_d = sum(epoch_d_loss) / len(epoch_d_loss)
        avg_g = sum(epoch_g_loss) / len(epoch_g_loss)
        avg_dx = sum(epoch_d_real) / len(epoch_d_real)
        avg_dgz = sum(epoch_d_fake) / len(epoch_d_fake)

        history['d_loss'].append(avg_d)
        history['g_loss'].append(avg_g)
        history['d_real'].append(avg_dx)
        history['d_fake'].append(avg_dgz)

        print(f"Epoch [{epoch+1:2d}/{n_epochs}] | "
              f"D: {avg_d:.4f} | G: {avg_g:.4f} | "
              f"D(x): {avg_dx:.3f} | D(G(z)): {avg_dgz:.3f}")

        # -- Early stopping signal ---------------------------------
        # If D(G(z)) < 0.05 for 3+ epochs: G is failing - investigate
        if avg_dgz < 0.05:
            print(f"  WARNING: D(G(z))={avg_dgz:.3f} < 0.05 - "
                  f"possible mode collapse or D dominance!")

        # If D loss < 0.1: D is too perfect - consider reducing LR or
        # increasing G update frequency
        if avg_d < 0.1:
            print(f"  WARNING: D Loss={avg_d:.4f} < 0.1 - "
                  f"Discriminator too dominant!")

    return history

# -- Usage ------------------------------------------------------
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Build Generator (from Section 2) and stable Discriminator
G = Generator(noise_dim=100, n_features=64).to(DEVICE)
D = StableDiscriminator(n_features=64).to(DEVICE)
```

```
G.apply(weights_init)
# Note: spectral_norm layers handle their own initialization

history = train_stable_gan(G, D, dataloader, n_epochs=25, device=DEVICE,
                           use_label_smooth=True, use_replay=True)

# Output:
# Epoch [ 1/25] | D: 1.2341 | G: 1.4523 | D(x): 0.823 | D(G(z)): 0.213
# Epoch [ 5/25] | D: 0.9812 | G: 1.0234 | D(x): 0.712 | D(G(z)): 0.342
# Epoch [10/25] | D: 0.8234 | G: 0.8912 | D(x): 0.634 | D(G(z)): 0.421
# Epoch [15/25] | D: 0.7541 | G: 0.7823 | D(x): 0.582 | D(G(z)): 0.463
# Epoch [25/25] | D: 0.7123 | G: 0.7412 | D(x): 0.531 | D(G(z)): 0.489
```

Intermediate Example – Training Diagnostics Dashboard

```
import matplotlib.pyplot as plt

def plot_training_dashboard(history):
    """
    Visualize all key GAN training metrics in one dashboard.
    Call after training: plot_training_dashboard(history)
    """
    epochs = range(1, len(history['d_loss']) + 1)

    fig, axes = plt.subplots(1, 3, figsize=(16, 4))

    # ── Plot 1: Loss curves ──────────────────────────────────
    ax1 = axes[0]
    ax1.plot(epochs, history['d_loss'], color='#C62828', linewidth=2,
↪   label='D Loss')
    ax1.plot(epochs, history['g_loss'], color='#1565C0', linewidth=2,
↪   label='G Loss')
    ax1.axhline(0.693, color='#F57C00', linewidth=1.5,
                linestyle='--', label='Equilibrium (ln 2)')
    ax1.set_title('Training Losses', fontsize=12, fontweight='bold')
    ax1.set_xlabel('Epoch'); ax1.set_ylabel('Loss')
    ax1.legend(fontsize=10); ax1.grid(True, alpha=0.3)

    # ── Plot 2: Discriminator scores ─────────────────────────
    ax2 = axes[1]
    ax2.plot(epochs, history['d_real'], color='#2E7D32', linewidth=2,
↪   label='D(x) - real')
    ax2.plot(epochs, history['d_fake'], color='#6A1B9A', linewidth=2,
↪   label='D(G(z)) - fake')
    ax2.axhline(0.5, color='#F57C00', linewidth=1.5,
                linestyle='--', label='Equilibrium = 0.5')
    ax2.fill_between(epochs,
                     [v - 0.05 for v in history['d_fake']],
                     [v + 0.05 for v in history['d_fake']],
                     alpha=0.2, color='#6A1B9A')
    ax2.set_title('Discriminator Scores Over Training', fontsize=12,
↪   fontweight='bold')
```

```
    ax2.set_xlabel('Epoch'); ax2.set_ylabel('D Output')
    ax2.legend(fontsize=10); ax2.grid(True, alpha=0.3); ax2.set_ylim(0, 1)

    # ── Plot 3: Health indicator (D(x) - D(G(z)) gap) ────────────
    ax3 = axes[2]
    gap = [dx - dgz for dx, dgz in zip(history['d_real'],
 ↪  history['d_fake'])]
    ax3.plot(epochs, gap, color='#F57C00', linewidth=2, label='D(x) -
 ↪  D(G(z)) gap')
    ax3.axhline(0.0, color='gray', linewidth=1.5, linestyle='--',
 ↪  label='Ideal: gap → 0')
    ax3.fill_between(epochs, gap, 0, alpha=0.2,
                     color=['#C62828' if g > 0.4 else '#2E7D32' for g in
 ↪  gap])
    ax3.set_title('D Confidence Gap\n(0 = equilibrium, high = D
 ↪  dominates)', fontsize=12, fontweight='bold')
    ax3.set_xlabel('Epoch'); ax3.set_ylabel('Score Gap')
    ax3.legend(fontsize=10); ax3.grid(True, alpha=0.3)

    plt.tight_layout()
    # Output: 3-panel dashboard showing loss convergence, D score balance,
    #         and the confidence gap shrinking toward 0 as training
 ↪  stabilizes

plot_training_dashboard(history)
```

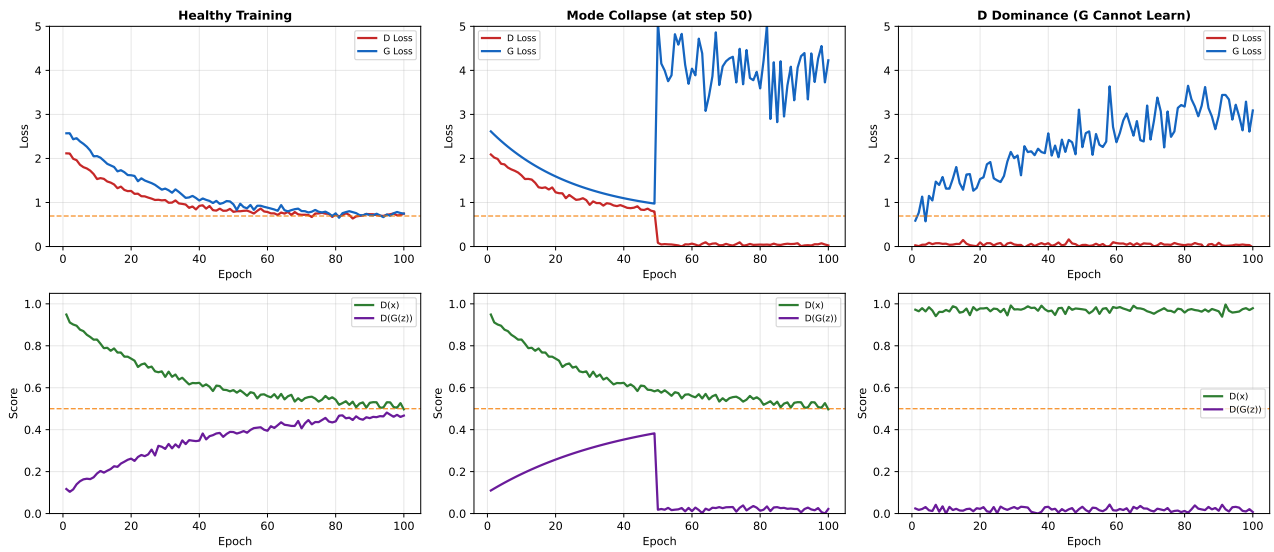
Common Errors and Edge Cases

```
# ---------------------------------------------------------------------
# ERROR 1: Not zeroing gradients at the right time
# ---------------------------------------------------------------------
# WRONG: Zeroing G gradients before the D update
G.zero_grad()           # ← This has no effect here - D update doesn't use
 ↪  G grads
D.zero_grad()
d_loss.backward()       # D grads accumulate from previous step too!
opt_D.step()

# CORRECT: Zero each optimizer's gradients immediately before its
 ↪  backward pass
D.zero_grad()           # Clear D grads right before D backward
d_loss.backward()
opt_D.step()

G.zero_grad()           # Clear G grads right before G backward
g_loss.backward()
opt_G.step()

# ---------------------------------------------------------------------
# ERROR 2: Checking G quality using D loss alone
# ---------------------------------------------------------------------
```

```python
# WRONG: Concluding G is doing well just because G loss is low
if g_loss < 0.5:
    print("Generator is improving!")  # ← NOT necessarily true

# CORRECT: Use D(G(z)) as the primary diagnostic for Generator quality
# D(G(z)) close to 0.5 means D cannot distinguish G's output from real
# D(G(z)) near 0 means G is failing; D(G(z)) near 1 means D has collapsed

with torch.no_grad():
    d_on_fake = D(G(torch.randn(64, 100, device=device))).mean().item()
print(f"D(G(z)) = {d_on_fake:.3f}")  # Target: close to 0.5

# ------------------------------------------------------------------
# ERROR 3: Using the same noise batch across the entire training step
# ------------------------------------------------------------------
noise = torch.randn(B, 100, device=device)
fake1 = G(noise)
D(fake1.detach())        # D update
g_loss = criterion(D(G(noise)), real_labels)  # WRONG: same noise - subtle
 ↪  correlation

# CORRECT: Fresh noise for Generator update step
noise_d = torch.randn(B, 100, device=device)
fake_for_D = G(noise_d).detach()
d_loss_fake = criterion(D(fake_for_D), fake_labels)

noise_g = torch.randn(B, 100, device=device)   # Fresh noise
fake_for_G = G(noise_g)
g_loss = criterion(D(fake_for_G), real_labels)
```

---

8. Comprehensive Diagnostics Reference



Reading the 6-panel dashboard:

- **Column 1 (Healthy):** Both losses converge toward $\ln 2$. $D(x)$ and $D(G(z))$ both converge toward 0.5. This is the ideal training signature.
- **Column 2 (Mode Collapse):** Until step 50, training looks healthy. Then at step 50, D loss crashes to near 0 (D became too accurate) and G loss spikes upward (G can no longer fool D). $D(G(z))$ drops to near 0. This is the diagnostic signature of collapse.
- **Column 3 (D Dominance):** D loss is near 0 from the start. G loss keeps rising – the Generator is receiving no useful signal. $D(G(z))$ is near 0 throughout. $D(x)$ is near 1. Action needed: reduce D's learning rate, add spectral norm, or reduce D update frequency.

---

9. Best Practices

- **Use spectral normalization on all Discriminator layers**: It is lightweight (adds ~5% compute overhead) and dramatically stabilizes training by preventing D from becoming overly sharp. It replaces the need for BatchNorm in the Discriminator.
- **Always monitor $D(x)$ and $D(G(z))$, not just the losses**: These two numbers give a clearer picture of the training state than the losses alone. Both should trend toward 0.5.
- **Implement early warning checks**: Log a warning when $D(G(z)) < 0.05$ (Generator failure) or $\mathcal{L}_D < 0.1$ (Discriminator dominance). Catching these early allows you to adjust hyperparameters before training is completely wasted.
- **Use experience replay when training on complex multi-modal data**: It adds minimal overhead (a deque of tensors) and provides substantial protection against mode cycling.
- **Save model checkpoints every 5 epochs and generated sample grids every epoch**: The best model is often not the final model. Checkpointing lets you recover from sudden quality degradation.
- **Use progressive growing for resolutions above 128×128**: Training a 256×256 GAN from scratch is unreliable. Progressive growing consistently produces better results with fewer training instabilities.

Quick Reference

```python
# Spectral normalization - wrap any layer
from torch.nn.utils import spectral_norm
layer = spectral_norm(nn.Conv2d(in_ch, out_ch, 4, 2, 1))

# Experience replay buffer
buffer = deque(maxlen=500)
buffer.extend(fake_imgs.detach().cpu())        # Store fakes
historical = torch.stack(random.sample(buffer, B))  # Retrieve mix

# Label smoothing (one-sided)
real_labels = torch.full((B, 1), 0.9, device=device)   # Not 1.0
fake_labels = torch.zeros(B, 1, device=device)          # Keep 0.0

# Key diagnostics - check every epoch
d_on_real = D(real_imgs).mean().item()          # Target: → 0.5
d_on_fake = D(G(noise)).mean().item()           # Target: → 0.5
```

```
gap = d_on_real - d_on_fake                    # Target: → 0.0

# Warning thresholds
if d_on_fake < 0.05: print("WARNING: G failing or mode collapse")
if d_loss < 0.10:    print("WARNING: D dominant - Generator receives no
 ↪  gradient")
```